

Servidor MCP para PCA: Verificación, Permisos y Valor del Protocolo

Araceli Castillo

Maestría en Inteligencia Artificial, Atlantis University

MAI 540: Machine Learning

Prof. Kevin A. García Gallardo

6 de octubre de 2026

Servidor MCP para PCA: Verificación, Permisos y Valor del Protocolo

En clase se completó `mcp_server.py`, un servidor del Protocolo de Contexto de Modelos (MCP) que expone análisis de componentes principales (PCA) sobre los archivos de la carpeta `datasets/`. Este informe documenta cómo se verificó cada pieza, el flujo completo entre el asistente y el servidor, los permisos que concede y qué aporta el protocolo frente a llamar las funciones directamente.

Verificación de las Cinco Piezas

Cada pieza se completó en su propio commit y se probó en el MCP Inspector antes de pasar a la siguiente. La tool `cargar_dataset` con `iris` devolvió 150 filas, cuatro columnas numéricas y una categórica. La tool `ejecutar_pca` con `iris` y dos componentes devolvió una varianza de 72.96 % y 22.85 %, en total 95.81 %, con las cargas de cada variable, calculadas con `scikit-learn` (Pedregosa et al., 2011). El resource `data://datasets` devolvió la lista `["iris", "wine"]`, y el resource con plantilla `data://datasets/wine` devolvió 178 filas y 13 columnas numéricas. El prompt `interpretar_componentes` devolvió un mensaje con los tres pasos de interpretación y la pregunta final sobre la varianza acumulada. Las mismas pruebas se repitieron por protocolo con un cliente propio, y la prueba final se hizo en mi computadora desde una terminal nueva.

Flujo de la Integración

Al conectarse, el cliente (el Inspector, Claude Desktop o el CLI del proyecto) pide la lista de tools, resources y prompts. Los resources los lee el cliente directamente: con `data://datasets` sabe qué archivos hay, sin que el modelo decida nada. Las tools las decide el asistente a partir de su descripción. Primero llama a `cargar_dataset` para conocer las columnas numéricas y cuántos componentes puede pedir, y después a `ejecutar_pca` con el número de componentes. La herramienta externa devuelve un JSON con la varianza por componente, la varianza acumulada y

las cargas, nunca las filas. Ese resultado se incorpora al proyecto mediante el prompt interpretar_componentes, que pide identificar las variables con mayor carga, explicar el patrón que representan y decidir si los componentes alcanzan.

Lo que Encontró la Verificación

Probar las piezas una por una reveló dos defectos, corregidos en commits aparte y documentados con capturas de antes y después. Primero, las dos tools aceptaban un nombre con "../" y leían cualquier archivo CSV del disco al que llegara el proceso; el resource con plantilla no tenía el problema porque el SDK rechaza esa dirección. Ahora los nombres pasan por una lista de permitidos que solo acepta los archivos de datasets/. Segundo, cuando una tool fallaba, el cliente solo veía «Error executing tool», sin el motivo, y el asistente no podía corregir su siguiente llamada. Ahora el error explica qué datasets existen o en qué rango debe estar el número de componentes.

Tabla 1
Permisos del servidor y su justificación

Aspecto	Detalle
Puede	Leer, sin escribir, los .csv de datasets/: nombres, número de filas, columnas y resultado del PCA.
No expone	Archivos fuera de datasets/ (lista de permitidos en _ruta_dataset); filas individuales (solo agregados; pca_utils.cargar_dataset, que devuelve el DataFrame completo, no está decorada); internet (sin librerías de red, transporte stdio); la clave de la API (el servidor solo recibe HOME, LC_CTYPE, PATH, SHELL y TERM).
Por qué es el mínimo	Cada permiso corresponde a una pieza: listar nombres, describir un CSV y ajustar el PCA. Ninguna necesita escribir, salir de datasets/ ni ver registros.
Límite	La restricción está en el código, no en el sistema operativo; el protocolo no la impone por sí mismo (Model Context Protocol, 2025). Un CSV sensible copiado a datasets/ quedaría expuesto.

Qué Aporta MCP Frente a Importar pca_utils.py

Al importar pca_utils.py, mi código decide cuándo correr el PCA y con cuántos componentes; como servidor MCP decide Claude, guiado solo por las descripciones, y por eso la de cargar_dataset indica usarla antes de ejecutar_pca y los errores deben llegar con su motivo

para que corrija la siguiente llamada. El servidor, además, fija una frontera que el import no tiene: el cliente ve cinco piezas y no puede llamar a la función que devuelve los datos completos. Por último, el mismo archivo funcionó sin cambios en el Inspector y en un cliente propio, y el prompt evaluado viaja junto con las tools.

Referencias

Model Context Protocol. (2025). *Specification* (Versión 2025-11-25).

<https://modelcontextprotocol.io/specification/2025-11-25>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.